

iesolver v4 — Arařtırmacı Rehberi

Bu rehber; iesolver v4'ü hiç görmemiş bir arařtırmacı için yazılmıřtır. Sistemi nasıl alıřtıracalıınızı, LLM konuřmalarını nasıl izleyecealıınızı, prompt'ları nasıl geliřtirecealıınızı ve deney tasarımıını nasıl uygulayacalıınızı adım adım açıklar.

İindekiler

1. v3 ile v4 Farkı
2. Kurulum
3. Bir Problem Tanımlama
4. Programı Bařlatma
5. Ařama Ařama İlerleyiři İzleme
6. LLM Konuřmalarını Görme
7. Prompt'ları Nasıl Geliřtirilir
8. Deney Tasarımı
9. Test Suite
10. Dosya Referans Tablosu
11. Kodun Makale Uygunluđu — Deđerlendirme
12. Program Akıř Haritası
13. DSPy Optimizasyon Mantıđı
14. Makale Yol Haritası — Kodda Tamamlanacaklar

1. v3 ile v4 Farkı

Özellik	v3	v4
Mimari	Monolitik Python script	LangGraph DAG + DSPy node'lar
Çalıştırma	Tek fonksiyon çağırısı	<code>solve()</code> / <code>stream_solve()</code>
Akış izleme	Print statement'lar	<code>stream_solve()</code> → node bazlı yield
LLM geçmişi	Loglama	<code>show_llm_history()</code> + <code>lm.history</code>
Prompt düzenleme	Hardcoded f-string'ler	DSPy Signature'ları (<code>src/iesolver/signatures/</code>)
Prompt optimizasyonu	Manuel	MIPROv2 otomasyonu (<code>scripts/optimize_mipro.py</code>)
Rapor	Tek formatlı metin	HTML + DOCX + PDF (Faz 5)
Ablasyon testleri	Yok	A1-A5 bayrakları + ie-eval paketi
Veri desteği	CSV	CSV / XLSX / SQLite
Tekrarlanabilirlik	Yok	<code>temperature=0</code> , <code>seed=42</code> + checkpoint

v3'teki akışınız v4'te şu şekilde karşılanır:

```

v3: problem_tanımla() → çalıştır() → raporu_gör()
v4: stream_solve("problem metni", data_path="veri.csv")
    → her node tamamlandığında (node_adı, partial_state) yield eder
    → write_report(final_state, "rapor.html", format="html")

```

2. Kurulum

```

# Projenin kök dizininde:
cd "v4/"

# Temel bağımlılıklar + rapor modülü:
uv sync --extra report

# Tüm eklentilerle (geliştirme ortamı):
uv sync --all-extras

# API anahtarınızı ayarlayın (.env dosyası veya ortam değişkeni):
# .env dosyası oluşturun:
echo IESOLVER_API_KEY=sk-... > .env
echo IESOLVER_FAST_MODEL=gemini/gemini-2.0-flash >> .env
echo IESOLVER_REASONING_MODEL=gemini/gemini-2.0-flash >> .env

```

Desteklenen model formatları (LiteLLM üzerinden):

- gemini/gemini-2.0-flash
- openai/gpt-4o-mini
- anthropic/claude-3-5-haiku-20241022

3. Bir Problem Tanımlama

Sadece metin (veri yok)

```
problem = "EOQ problemi: Yıllık talep D=10000 birim, sipariş maliyeti S=50 TL, birim tutma maliyeti H=
```

Veri dosyasıyla

```
problem = "Ekteki CSV'deki lokasyonlar için ulaşım maliyetini minimize eden taşıma planını bul."  
data     = "data/transport.csv"
```

Kabul edilen veri formatları:

- .csv → tek tablo (tables["data"])
- .xlsx → her sayfa ayrı tablo (tables["Sheet1"] , ...)
- .sqlite → her tablo ayrı (tables["orders"] , ...)

İyi bir problem metni nasıl olmalı?

Sistem G-O-C çerçevesini uygular: **G**oal (Hedef), **O**utput (Çıktı), **C**onstraint (Kısıt).

```
İyi: "D=10000, S=50, H=2 parametreleriyle EOQ modelini çöz. Optimal sipariş  
miktarını ve toplam yıllık maliyeti döndür."
```

```
Zayıf: "Stok optimizasyonu yap."
```

Metninizde hedef, beklenen çıktı ve kısıtlar açık değilse, sistem `clarify` node'unda durup ek bilgi ister (interaktif mod) veya varsayım yaparak devam eder (auto mod).

4. Programı Başlatma

A) Terminal üzerinden (en basit yol)

```
# Temel kullanım:
uv run python scripts/run_problem.py "EQ: D=10000, S=50, H=2"

# Veri dosyasıyla:
uv run python scripts/run_problem.py "LP problemi" --data data/input.csv

# PDF rapor:
uv run python scripts/run_problem.py "EQ problemi" --format pdf

# LLM geçmişini de göster:
uv run python scripts/run_problem.py "EQ" --show-llm-log

# Interrupt olmadan (batch/test modu):
uv run python scripts/run_problem.py "EQ" --auto

# Ablasyon testleri:
uv run python scripts/run_problem.py "EQ" --no-refiner # A1
uv run python scripts/run_problem.py "EQ" --no-retry # A2
uv run python scripts/run_problem.py "EQ" --fast-only # A4
```

B) Python'dan (notebook veya script)

```
from iesolver import solve, stream_solve, show_llm_history
from iesolver.report import write_report

# Basit çalıştırma (tüm akış tamamlanınca döner):
state = solve("EQ: D=10000, S=50, H=2")
print(state["executive_summary"])
print(state["execution_result"])

# Raporu kaydet:
write_report(state, "raporum.html", format="html")
write_report(state, "raporum.pdf", format="pdf")
write_report(state, "raporum.docx", format="docx")
```

C) Kesintili çalıştırma (checkpoint ile devam)

```
import uuid
thread = str(uuid.uuid4())

# İlk çalıştırma – interrupt olursa durur:
state = solve("LP optimizasyon", thread_id=thread, auto_mode=False)

# Eğer interrupt olduysa, kullanıcı cevabını ekleyip devam edin:
from langgraph.types import Command
# (LangGraph interrupt akışı – gelişmiş kullanım)
```

5. Aşama Aşama İlerleyişi İzleme

`stream_solve()`, her LangGraph node tamamlandığında `(node_adı, partial_state)` üretir. v3'teki adım adım görünümü buradan gelir.

Temel kullanım

```
from iesolver import stream_solve

for node_name, partial in stream_solve("EQ: D=10000, S=50, H=2"):
    keys = [k for k in partial if not k.startswith("_")]
    print(f" ✓ {node_name:<25} → {keys}")
```

Beklenen çıktı:

✓ intake	→ ['raw_prompt', 'data_bundle', 'data_summary']
✓ requirement	→ ['explicit_goal', 'constraints', 'is_complete']
✓ refiner	→ ['refined_prompt']
✓ route	→ ['execution_path']
✓ code_branch	→ ['generated_code', 'retry_count']
✓ validate	→ ['is_valid', 'validation_notes']
✓ sensitivity	→ ['sensitivity_results']
✓ artifacts	→ ['figures']
✓ report	→ ['executive_summary', 'technical_output', 'action_directives']

Belirli alanları yakalamak

```

final_state = {}
for node_name, partial in stream_solve("EOQ problemi"):
    final_state.update(partial)
    if node_name == "validate":
        print("Doğrulama:", partial.get("is_valid"), partial.get("validation_notes"))
    if node_name == "report":
        print(partial.get("executive_summary", "")[:300])

```

Node sırası ve açıklaması

Node	Ne yapar	Hangi alanlara yazar
intake	Ham prompt + veri dosyasını okur, özet çıkarır	data_bundle , data_summary
requirement	G-O-C çerçevesiyle gereksinimler çıkarır	explicit_goal , constraints , is_complete
clarify	Eksik bilgi varsa interrupt/soru üretir	clarification_questions
refiner	Prompt'u yapısal DSPy formatına dönüştürür	refined_prompt
route	CODE vs NO_CODE kararı verir	execution_path
code_branch	Algoritma seçer, kod üretir (ReAct)	generated_code , algo_name
validate	Kodu çalıştırır, sonucu doğrular	is_valid , execution_result
sensitivity	Duyarlılık analizi yapar	sensitivity_results
artifacts	Grafik/figür oluşturur	figures
chain_branch	NO_CODE yolu: analitik çözüm	technical_output
report	3-bölümlü rapor yazar	executive_summary , technical_output , action_directives

6. LLM Konuşmalarını Görme

Her LLM çağrısı `lm.history` listesine eklenir. `show_llm_history(n)` son `n` çağrışı ayrıntılı basar.

Temel kullanım

```
from iesolver import solve, show_llm_history

state = solve("EQ: D=10000, S=50, H=2")
show_llm_history(n=5) # Son 5 LLM çağrısını göster
```

Çıktı formatı:

```
[1/5] Model: gemini/gemini-2.0-flash
Tokens: 342 giriş / 87 çıkış
```

— SYSTEM —

You are a meticulous Industrial Engineering Requirement Analyst...

— USER —

Cleaned prompt: EQ: D=10000, S=50, H=2...

Data summary: (veri yok)

— ASSISTANT —

```
{"is_complete": true, "explicit_goal": "Minimize total inventory cost..."}
```

Terminal üzerinden (run_problem.py ile)

```
uv run python scripts/run_problem.py "EQ" --show-llm-log --llm-log-n 10
```

Doğrudan history'ye erişim

```
from iesolver.lm import get_fast_lm, get_reasoning_lm

fast_lm = get_fast_lm()
reasoning_lm = get_reasoning_lm()

# Her entry: {"model": "...", "messages": [...], "response": {...}, "usage": {...}}
for entry in fast_lm.history[-3:]:
    print(entry["model"], entry.get("usage", {}))
```

7. Prompt'ları Nasıl Geliştirilir

Hangi dosyayı düzenlemeliyim?

Tüm LLM talimatları `src/iesolver/signatures/` klasöründedir. Her `.py` dosyası bir DSPy signature sınıfı içerir.

Signature dosyası	Hangi node kullanır	Ne kontrol eder
<code>gatekeeper.py</code>	<code>intake</code>	Ham prompt temizleme, veri özetleme
<code>requirement_analyst.py</code>	<code>requirement</code>	G-O-C çerçevesi, eksik bilgi tespiti
<code>prompt_refiner.py</code>	<code>refiner</code>	Yapısal prompt dönüştürme
<code>strategy_router.py</code>	<code>route</code>	CODE/NO_CODE kararı mantığı
<code>algo_selector.py</code>	<code>code_branch/algo_select</code>	Algoritma seçim kriterleri
<code>constraint_adapter.py</code>	<code>code_branch/constraint_adapt</code>	Kısıt tanımlama
<code>output_spec.py</code>	<code>code_branch/output_spec</code>	Beklenen çıktı formatı
<code>react_code.py</code>	<code>code_branch/generate</code>	Kod üretimi (ReAct döngüsü)
<code>validator.py</code>	<code>validate</code>	Doğrulama kriterleri
<code>sensitivity.py</code>	<code>sensitivity</code>	Duyarlılık analizi yönergesi
<code>final_report.py</code>	<code>report</code>	Rapor formatı ve tonu
<code>analytical_engine.py</code>	<code>chain_branch</code>	NO_CODE analitik çözüm

Signature nasıl düzenlenir?

```
# src/iesolver/signatures/requirement_analyst.py

class RequirementAnalystSignature(dspy.Signature):
    """
    [BURASI SYSTEM PROMPT – LLM'e ne yapması gerektiği]
    DSPy bu docstring'i system prompt olarak gönderir.
    """
    # InputField'lar: node'dan gelen veriler
    cleaned_prompt: str = dspy.InputField(desc="...")
    data_summary: str = dspy.InputField(desc="...")

    # OutputField'lar: LLM'in döndürmesi beklenenler
    is_complete: bool = dspy.OutputField(desc="...")
    explicit_goal: str = dspy.OutputField(desc="...")
    constraints: list[str] = dspy.OutputField(desc="...")
```

Düzenleme noktaları:

1. `"""..."""` docstring → sistem direktifi (en etkili)

2. InputField(desc="...") → LLM'e alan açıklaması
3. OutputField(desc="...") → beklenen format açıklaması

Değişikliği test etme

```
# Signature'ı düzenledikten sonra:  
uv run python scripts/run_problem.py "EQ: D=10000, S=50, H=2" --show-llm-log
```

Çıktıda değişen system prompt'u ve LLM'in yeni yanıtını görürsünüz.

Otomatik Prompt Optimizasyonu (MIPROv2)

DSPy'in MIPROv2 algoritması, Signature prompt'larını otomatik olarak optimize edebilir:

```
# Pilot koşu (düşük maliyet):  
uv run python scripts/optimize_mipro.py \  
  --train-data data/nl4opt_train_cleaned.jsonl \  
  --output compiled/iesolver_mipro.json \  
  --max-train 40 --num-candidates 5 --num-trials 10  
  
# Kurulum doğrulama (LLM çağrısı yok):  
uv run python scripts/optimize_mipro.py \  
  --train-data data/nl4opt_train_cleaned.jsonl \  
  --output compiled/test.json --dry-run
```

Optimize edilmiş program JSON olarak kaydedilir ve

iesolver._optimization.load_compiled_graph(path) veya
ie_eval.ablations.make_a5_solve(compiled_path=path) ile yüklenir. Ayrıntılı mantık için bkz. §13 DSPy Optimizasyon Mantığı. Bu, makalenin ana araştırma katkısıdır (A5 ablasyonu).

8. Deney Tasarımı

Ablasyon Bayrakları (EVALUATION_PLAN §5)

Ablasyon	Bayrak / Mekanizma	Terminal	Neyi ölçer
A1	<code>solve(..., enable_refiner=False)</code>	<code>--no-refiner</code>	PromptRefiner katkısı
A2	<code>solve(..., enable_validator_retry=False)</code>	<code>--no-retry</code>	Retry döngüsü katkısı
A3	<code>ie_eval.ablations.make_a3_correctness_fn()</code> — deterministik doğrulama kapatılır, yalnız LLM validator sinyali kullanılır	— (ie-eval)	Deterministik doğrulama katmanı katkısı
A4	<code>solve(..., fast_only=True)</code>	<code>--fast-only</code>	Reasoning LM anahtarlama katkısı
A5	<code>ie_eval.ablations.make_a5_solve(compiled_path)</code> — MIPROv2 optimize Signature'lar	— (ie-eval)	MIPROv2 optimizasyonunun katkısı

Otoritatif tanımlar: `ie-eval/src/ie_eval/ablations.py` (EVALUATION_PLAN §5 ile hizalı).

Toplu Değerlendirme (ie-eval paketi)

```
cd ie-eval/
uv run python -m ieeval.runner \
  --dataset datasets/nl4opt_sample.jsonl \
  --config configs/full.yaml \
  --out results/

# Sadece ablasyon A1 (refiner yok):
uv run python -m ieeval.runner --config configs/ablation_a1.yaml
```

Metrikler

Sistem her node için otomatik olarak toplar:

```
state = solve("EQQ problemi")
print(state["metrics"])
# {
#   "intake":  {"llm_calls": 1, "tokens_in": 342, "tokens_out": 87,
#               "cost_usd": 0.0001, "latency_ms": 450},
#   "report":  {"llm_calls": 1, "tokens_in": 800, "tokens_out": 250, ...},
#   ...
# }
```

Raporlarda "Performance Metrics" tablosu olarak otomatik eklenir.

Tekrarlanabilirlik

Tüm çalıştırmalar `temperature=0`, `seed=42` ile yapılır (`.env` ile değiştirilebilir). LangGraph checkpoint SQLite'a kaydedilir (`checkpoints/solver.db`). Aynı `thread_id` ile çağrı yapılırsa önceki durum yüklenir.

```
# Aynı thread_id ile ikinci çalıştırma → checkpoint'ten devam
state2 = solve("EOQ", thread_id="benim-thread-123")
```

9. Test Suite

```
# Tüm testler:
uv run pytest

# Sadece rapor testleri:
uv run pytest tests/test_report_writer.py -v

# Sadece Faz 4 (duyarlılık):
uv run pytest tests/test_faz4.py -v

# Yavaş testler hariç (LLM çağrısı yapmaz):
uv run pytest -m "not slow"

# ie-eval testleri:
cd ie-eval/ && uv run pytest
```

Test dosyası	Ne test eder
test_report_writer.py	HTML/DOCX/PDF üretimi (15 test)
test_faz4.py	Duyarlılık analizi, artifact üretimi
test_typed_signatures.py	DSPy Signature tip kontrolü
test_metrics.py	Node metrik toplama
test_auto_mode.py	Auto mode / interrupt davranışı
test_reproducibility.py	Seed/determinizm
test_e2e_eoq.py	Uçtan uca EOQ smoke testi (gerçek LLM)

Test sırasında üretilen dosyalar (PDF, DOCX, HTML) şuraya kaydedilir: c:\Users\
<kullanıcı>\AppData\Local\Temp\pytest-of-<kullanıcı>\pytest-NN\

10. Dosya Referans Tablosu

Giriş Noktaları

Dosya	Amaç
src/iesolver/__init__.py	Public API: <code>solve()</code> , <code>stream_solve()</code> , <code>show_llm_history()</code> , <code>write_report()</code>
scripts/run_problem.py	Terminal CLI: problem ver → aşama aşama izle → rapor al
scripts/optimize_mipro.py	MIPROv2 ile DSPy Signature'larını otomatik optimize et

Çekirdek Modüller

Dosya	Amaç
src/iesolver/config.py	Settings sınıfı: API key, model isimleri, izin yolları (<code>.env</code> 'den)
src/iesolver/state.py	SolverState (TypedDict) + DataBundle — pipeline'daki ortak veri
src/iesolver/graph.py	LangGraph DAG tanımı: node'lar, edge'ler, conditional routing
src/iesolver/lm.py	LM yönetimi: <code>call_with_fast_lm()</code> , <code>call_with_reasoning_lm()</code> , <code>call_with_configured_lm()</code>

Node'lar (İş Mantığı)

Dosya	Node	Görev
src/iesolver/nodes/intake.py	intake	Ham prompt temizleme + veri yükleme
src/iesolver/nodes/requirement.py	requirement	G-O-C gereksinim çıkarımı
src/iesolver/nodes/clarify.py	clarify	Eksik bilgi → interrupt veya varsayım
src/iesolver/nodes/refine.py	refiner	Prompt → yapısal DSPy formatı
src/iesolver/nodes/route.py	route	CODE / NO_CODE karar
src/iesolver/nodes/code_branch/algo_select.py	code_branch (1/4)	Algoritma seçimi
src/iesolver/nodes/code_branch/constraint_adapt.py	code_branch (2/4)	Kısıt adaptasyonu
src/iesolver/nodes/code_branch/output_spec.py	code_branch (3/4)	Çıktı tanımı
src/iesolver/nodes/code_branch/generate.py	code_branch (4/4)	ReAct kod üretimi
src/iesolver/nodes/validate.py	validate	Sandbox çalıştırma + doğrulama
src/iesolver/nodes/chain_branch.py	chain_branch	NO_CODE analitik çözüm
src/iesolver/nodes/sensitivity.py	sensitivity	Duyarlılık analizi
src/iesolver/nodes/artifacts.py	artifacts	Grafik/figür üretimi
src/iesolver/nodes/report.py	report	3-bölümlü rapor metni

DSPy Signature'ları (Prompt Mühendisliği)

Dosya	Kontrol ettiği prompt
src/iesolver/signatures/gatekeeper.py	Girdi temizleme direktifi
src/iesolver/signatures/requirement_analyst.py	G-O-C çerçevesi + tamamlık değerlendirmesi
src/iesolver/signatures/prompt_refiner.py	Yapısal dönüştürme direktifi
src/iesolver/signatures/strategy_router.py	CODE/NO_CODE karar mantığı
src/iesolver/signatures/algo_selector.py	Algoritma seçim kriterleri
src/iesolver/signatures/constraint_adapter.py	Kısıt formülasyon direktifi
src/iesolver/signatures/output_spec.py	Çıktı format direktifi
src/iesolver/signatures/react_code.py	ReAct döngüsü kod üretim direktifi
src/iesolver/signatures/validator.py	Doğrulama kriterlerini direktifi
src/iesolver/signatures/sensitivity.py	Duyarlılık analizi direktifi
src/iesolver/signatures/tornado_chart.py	Tornado chart veri direktifi
src/iesolver/signatures/analytical_engine.py	NO_CODE analitik çözüm direktifi
src/iesolver/signatures/final_report.py	Rapor bölümü yazım direktifi

Rapor Modülü

Dosya	Amaç
src/iesolver/report/__init__.py	ReportWriter, write_report export
src/iesolver/report/writer.py	ReportWriter sınıfı — format yönlendirmesi
src/iesolver/report/_html.py	Jinja2 + markdown-it-py → HTML renderer
src/iesolver/report/_docx.py	python-docx → DOCX renderer
src/iesolver/report/_pdf.py	fpdf2 → PDF renderer (saf Python, GTK gerekmez)
src/iesolver/report/templates/report.html.j2	HTML rapor Jinja2 şablonu

Altyapı

Dosya	Amaç
src/iesolver/io/data_loader.py	CSV/XLSX/SQLite → DataBundle dönüşümü
src/iesolver/sandbox/runner.py	Üretilen Python kodunu izole sandbox'ta çalıştırır
src/iesolver/observability/metrics.py	Node başına LLM çağrı sayısı, token, maliyet, gecikme
src/iesolver/_optimization.py	Optimize edilmiş DSPy programını yükleme/kaydetme

Değerlendirme (ie-eval paketi)

Dosya	Amaç
ie-eval/	Ayrı uv workspace paketi — benchmark runner
ie-eval/ieeval/runner.py	NL4Opt / IndustryOR dataset'leri üzerinde toplu değerlendirme

Konfigürasyon

Dosya	Amaç
.env	IESOLVER_API_KEY , IESOLVER_FAST_MODEL , IESOLVER_REASONING_MODEL
pyproject.toml	Proje bağımlılıkları, ruff, mypy, pytest ayarları
PLAN.md	5 fazlı yol haritası ve teknik kararlar
METHODOLOGY_NOTES.md	Makale için mimari kararlar ve gerekçeler
SYSTEM.md	Sistem mimarisinin tam AI-odaklı dokümantasyonu
EVALUATION_PLAN.MD	Deney tasarımı, metrikler, ablasyon planı
MAKALE_YOL_HARITASI.md	Kod sonrası → Q1 yayın iş listesi (18 iş)

11. Kodun Makale Uygunluğu — Değerlendirme

Son değerlendirme: 2026-07-04. Hızlı test suite: 54/54 geçti (`pytest -m "not slow"`).

Genel Yargı

Kod, makalenin metodoloji bölümünü taşıyacak olgunlukta. Mimari iddiaların (state-as-contract, bounded retry, ikili doğrulama, model tier anahtarlama, MIPROv2 optimizasyonu) hepsinin koda somut karşılığı var ve test ediliyor. Değerlendirmede bulunan tek gerçek tutarsızlık (Bulgu 1, A4 bayrağı) aynı gün düzeltildi; geri kalanlar bilgi notu düzeyinde.

Güçlü Yanlar (makale argümanlarını destekleyen)

- **Tip-güvenli state sözleşmesi** — `SolverState TypedDict`, her node yalnız yazdığı alanı döner; `LangGraph partial-merge`. Makaledeki "state-as-contract" argümanının birebir karşılığı (`state.py`).
- **İki katmanlı hata kurtarma** — iç katman: `ReAct max_iters=3` (kod yaz → çalıştır → hatayı gör → düzelt); dış katman: `validate → code_branch` retry döngüsü (`MAX_RETRIES=3`). "Autonomous Error Recovery with Bounded Retries."
- **Tekrarlanabilirlik** — `temperature=0, seed=42` her LM örneğine uygulanıyor; `SQLite checkpoint; test_reproducibility.py` bunu doğruluyor.
- **Telemetri** — her node'da `@instrument` ile token/maliyet/gecikme toplama; `merge_metrics reducer` retry geçişlerini `invocations` ile sayıyor. Makalenin maliyet-doğruluk analizinin ham verisi buradan geliyor.
- **Ablasyon altyapısı** — A1/A2/A4 bayrak olarak state'te, A3/A5 ie-eval tarafında; kod kopyalamadan config ile koşulabiliyor (yol haritası İş 8'in şartı).
- **MIPROv2 entegrasyonu** — singleton paylaşımı tasarımı (`_optimization.py`) doğru kurgulanmış; `save/load JSON` ile "ham vs optimize" iki konfigürasyon temiz şekilde ayrılıyor.

Bulgular

Bulgu 1 (✅ düzeltildi, 2026-07-04): `fast_only=True` bayrağını (A4) yalnızca `generate.py` ve `sensitivity.py` dikkate alıyordu; şu dört node bayrağı yok sayıp doğrudan `call_with_reasoning_lm` çağırıyordu: `algo_select.py`, `constraint_adapt.py`, `output_spec.py`, `validate.py`. Yani A4 koşusunda reasoning LM "kapalı" sanılırken 6 çağrının 4'ü hâlâ reasoning modeli kullanırdı — ablasyon geçersiz sonuç üretti. Dört çağrı da `call_with_configured_lm(module, fast_only=state.get("fast_only", False), ...)` biçimine çevrildi; artık reasoning-tier'daki 6 çağrının tamamı bayrağa uyuyor. Doğrulama: v4 54/54 + ie-eval 144/144 test geçti.

Bulgu 2 (karar noktası — İş 6): `MAKALE_YOL_HARITASI.md` İş 6, maliyet gerekçesiyle yalnızca 2 kritik Signature'ın (`AlgoSelector` + `ReAct`) optimize edilmesini öneriyor; mevcut `IESolverProgram` ise 11 modülün hepsini optimizasyona açıyor. İkisi de savunulabilir; nasıl kısıtlanacağı §13.6'da.

Bulgu 3 (bilgi): `ie-eval/datasets/` ve `compiled/` henüz boş — bu hata değil, yol haritası İş 3, 4 ve 6'nın çıktıları. Adaptör kodları (`n14opt.py`, `industryor.py`, `ie_case.py`) hazır ve test edilmiş.

Bulgu 4 (giderildi): Bu rehberin §7 ve §8'indeki eski MIPROv2 komutu ve A3/A5 tanımları kodla çelişiyordu; bu güncellemede `optimize_mipro.py` ve `ablations.py` ile hizalandı.

12. Program Akış Haritası

Kuşbakışı: bir problemin yaşam döngüsü

solve("EQ: D=10000, S=50, H=2") çağrıldığında olan biten, tek cümleyle: **ham metin → temizle → gereksinimleri çıkar → (eksikse sor) → prompt'u refine et → çözüm yolunu seç → kodu üret ve çalıştır → doğrula (gerekirse tekrar dene) → duyarlılık analizi → grafikler → 3 katmanlı rapor.**



Node × LM katmanı × State sözleşmesi

Her node'un hangi modeli kullandığı ve state'e ne yazdığı — prompt geliştirirken "hangi Signature'a dokunursam neyi etkilerim" sorusunun cevabı:

Node	DSPy modülü (tipi)	LM	Okur	Yazar
intake	GateKeeper (CoT)	fast	raw_prompt , data_path	cleaned_prompt , data_bundle , data_summary
requirement	RequirementAnalyst (Predict)	fast	cleaned_prompt , data_summary	is_complete , missing_items , explicit_goal , constraints , output_spec
clarify	— (LLM yok)	—	missing_items , auto_mode	user_clarification / auto_assumptions_log
refine	PromptRefiner (CoT)	fast	G-O-C alanları	essential_prompt , strict_constraints , problem_type
route	StrategyRouter (CoT)	fast	essential_prompt , problem_type	execution_path , rationale
code_branch 1/4	AlgoSelector (CoT)	reasoning	essential_prompt , problem_type , data_summary	target_algorithm , target_library
code_branch 2/4	ConstraintAdapter (Predict)	reasoning	strict_constraints , target_library	library_specific_constraints
code_branch 3/4	OutputSpecEngineer (Predict)	reasoning	output_spec , target_library	code_output_spec
code_branch 4/4	ReActCodeGenerator (ReAct + sandbox tool)	reasoning	yukarıdakilerin hepsi	final_code , execution_result , retry_count +1
validate	ResultValidator (CoT)	reasoning	essential_prompt , strict_constraints , execution_result	is_valid , confidence_score , validation_notes
sensitivity	SensitivityAnalysis (CoT)	reasoning	çözüm + parametreler	sensitivity_results
artifacts	TornadoChart (Predict)	fast	sensitivity_results	figures
chain_branch	AnalyticalEngine (CoT)	fast	essential_prompt	raw_result , solution_path
report	FinalReportGenerator (CoT)	fast	tüm sonuç alanları	executive_summary , technical_output , action_directives

Okuma önerisi: Akışa hâkim olmak için sırasıyla `graph.py` (topoloji + conditional edge fonksiyonları, dosyanın başındaki ASCII şema), `state.py` (alan grupları faz faz), sonra tek tek node dosyaları. Her node dosyasının docstring'i "Reads / Writes" bölümü içeriyor.

Retry semantiği (sık karışan nokta)

- **İç döngü** (`generate.py`): ReAct, sandbox'tan hata görürse aynı LLM oturumu içinde en fazla 3 kez kodu düzeltir. Bu, `validate`'e hiç gitmeden olur.
- **Dış döngü** (`graph.py: _route_after_validate`): `validate is_valid=False` derse ve `retry_count < 3` ise `code_branch` **baştan** koşulur (algoritma seçimi dahil). `retry_count` her `code_branch` geçişinde +1 artar → toplam en fazla 3 `code_branch` geçişi. Hak bitince rapor yine de yazılır (başarısızlık raporu — makalede "graceful degradation").

13. DSPy Optimizasyon Mantığı

Bu bölüm, sistemde kurgulanmış optimizasyon mantığını sıfırdan öğretir. Amaç: İş 6'yı (MIPROv2 koşusu) bilinçli parametrelerle yürütebilmek ve makalede "neden DSPy" sorusunu savunabilmek.

13.1 Temel fikir: prompt bir string değil, derlenebilir bir program

Klasik yaklaşımda prompt elle yazılmış bir f-string'dir; iyileştirmek = elle deneme-yanılma. DSPy'da ise üç ayrı katman var:

1. **Signature** — *ne* istediğinin bildirimsel tanımı. Docstring = sistem talimatı; `InputField / OutputField` desc'leri = alan talimatları; tip anotasyonları (`bool`, `list[str]`) = çıktı formatı zorlaması. Projede: `src/iesolver/signatures/*.py` (13 dosya).
2. **Module** — *nasıl* çıkarım yapılacağı: `Predict` (tek atış), `ChainOfThought` (önce gerekçe üretir), `ReAct` (düşün → araç çağır → gözlemle döngüsü). Projede her node bir modül singleton'ı kurar, Örn. `_validator = dspy.ChainOfThought(ResultValidatorSignature)`.
3. **Optimizer (teleprompter)** — Signature'daki talimat metnini ve few-shot örnekleri *otomatik arayan* katman. Elle prompt mühendisliği yerine: metrik + eğitim verisi → en iyi talimat kombinasyonu.

Bu ayırım makalenin metodolojik iddiası: prompt'lar sistemin parametreleridir ve veriyle optimize edilebilir — model değişince yeniden derlenir (RQ3, model-agnostiklik bunun üstüne kurulu).

13.2 Optimizasyonun üç bileşeni bu projede nerede?

Bileşen	Ne	Projede
Program	Optimize edilecek DSPy modülleri	IESolverProgram (_optimization.py) — 11 modül singleton'ını attribute olarak tutar; forward(prompt) tüm pipeline'ı auto_mode=True ile koşar
Metrik	Başarı ölçüsü, (example, pred) → 0/1	correctness_metric (optimize_mipro.py) — execution_result içindeki sayı, ground truth'a numerical_match ile toleranslı karşılaştırılır
Trainset	Etiketli örnekler	NL4Opt train split JSONL → dspy.Example(prompt=..., optimal_value=...)

13.3 Kritik tasarım: singleton referans paylaşımı

En zarif nokta ve makalede vurgulanmaya değer:

```
nodes/validate.py:      _validator = dspy.ChainOfThought(...)  ← canlı nesne
_optimization.py:      self.validator = _validator             ← AYNI nesne
```

IESolverProgram modülleri **kopyalamaz**, aynı Python nesnelerine referans tutar. MIPROv2 self.validator 'ın prompt'unu güncellediğinde, pipeline'daki call_with_reasoning_lm(_validator, ...) çağrısı da otomatik olarak yeni prompt'u kullanır. Böylece:

- Optimizasyon sırasında forward() = gerçek solve() → **uçtan uca (end-to-end) değerlendirme**: bir ara-modülün prompt değişikliği, nihai sayısal doğruluk üzerinden puanlanır.
- program.save(path) → talimatlar + few-shot demolar JSON'a yazılır.
- load_compiled_graph(path) → aynı singleton'lara geri yüklenir; LangGraph yeniden derlenmez, solve() artık optimize prompt'larla çalışır.

(Yan etki: singleton'lar paylaşıldığı için optimize_mipro.py bilinçli olarak num_threads=1 kullanır — paralel deneme thread'leri aynı nesnenin prompt'unu ezerdi.)

13.4 MIPROv2 içeride ne yapar? (üç aşama)

- Bootstrapping (demo toplama)**: Mevcut (ham) programla trainset koşulur. Metriği geçen koşuların izlerinden (trace) her modül için aday few-shot örnekler toplanır — "başarılı geçmiş, örnek olur." (--max-bootstrapped-demos , --max-labeled-demos bunların üst sınırı.)
- Talimat önerisi (instruction proposal)**: Teacher LM (bizde reasoning model veya --teacher-model), her modül için --num-candidates adet alternatif sistem talimatı yazar. Bunu yaparken program yapısına, veri özetine ve 1. aşamadaki demolara bakar. Teacher'ın temperature=0.7 olması kasıtlı: aday çeşitliliği gerekir.

3. **Bayes arama:** `--num-trials` deneme boyunca "modül × talimat × demo" kombinasyonları trainset (mini-batch) üzerinde metrikle skorlanır; Bayesian optimizasyon umut vadeden bölgeye yoğunlaşır. En iyi kombinasyon kazanır ve `compiled/*.json` 'a yazılır.

Kredi ataması (credit assignment) nasıl çözülüyor? 11 modüllü zincirde "hangi prompt kötü" sorusunu MIPROv2 açıkça cevaplamaz; bunun yerine arama uzayında farklı kombinasyonları dener ve *nihai* metriği yükselten kombinasyonları seçer. Bu yüzden metrik uçtan uca doğruluk olmalı — ara adım metriği koymaya çalışmak (ör. "algoritma seçimi doğru mu") hem etiket gerektirir hem de pipeline-level optimizasyonun avantajını bozar.

13.5 Maliyet sezgisi ve parametre reçetesi

Kaba formül: $\text{LLM çağırısı} \approx \text{num_trials} \times \text{minibatch} \times (\text{problem başına} \sim 8 \text{ pipeline çağırısı}) + \text{teacher çağırısı}$.

Senaryo	Parametreler	Kaba maliyet (Flash sınıfı)
Dry-run	<code>--dry-run</code>	\$0 (LLM çağırısı yok)
Pilot	<code>--max-train 40 --num-candidates 5 --num-trials 10</code>	< \$2
Tam (makale)	<code>--num-candidates 15 --num-trials 30 --dev-data ...</code>	< \$10; Pro-teacher ~20×

Kurallar (yol haritası "Kritik Kurallar" ile aynı):

- **Yalnızca train split** — NL4Opt test 289'una asla dokunma (data leakage).
- Optimizasyonun kendi maliyetini kaydet — makalede raporlanır.
- Optimizasyon = deney konfigürasyonudur: `compiled/*.json` sürümlenir, "ham vs optimize" iki ayrı koşu olarak raporlanır (A5).

13.6 Karar noktası: 11 modül mü, 2 kritik modül mü?

Yol haritası İş 6, maliyet için yalnız AlgoSelector + ReAct'in optimize edilmesini öneriyor; mevcut `IESolverProgram` 11 modülün hepsini açıyor. Kısıtlamak istersen mekanizma basit: MIPROv2 yalnızca programın **attribute olarak gördüğü** modülleri optimize eder; `forward()` ise her durumda tüm pipeline'ı koşar. Yani `IESolverProgram.__init__` içinde yalnız `self.selector` ve `self.react` bırakılırsa, diğer 9 modül ham prompt'larıyla çalışmaya devam eder ama optimize edilmez → arama uzayı ve teacher maliyeti ciddi düşer. Önerim: pilotu 11 modülle koş; süre/maliyet taşarsa 2 modüle indir ve bu kararı makalede gerekçelendir ("targeted optimization").

13.7 Prompt geliştirme ile optimizasyonun ilişkisi

Elle prompt iyileştirme (§7) ve MIPROv2 rakip değil, sıralı:

1. **Önce elle:** Signature docstring'lerini G-O-C netliğinde yaz — MIPROv2 ham talimatı "tohum" olarak kullanır; iyi tohum, iyi adaylar üretir.
 2. **Sonra otomatik:** MIPROv2 koşusu → A5 konfigürasyonu.
 3. **Code freeze'den sonra** Signature'lara dokunulmaz; değişiklik = etkilenen tüm koşuların tekrarı.
-

14. Makale Yol Haritası — Kodda Tamamlanacaklar

MAKALE_YOL_HARITASI.md 'deki 18 işten kod tarafına dokunanlar. Durum sütunu 2026-07-04 itibarıyla.

İş	Kodda yapılacak	İlgili dosyalar	Durum
1. Sistem incelemesi	A4 düzeltmesi  (2026-07-04) + IMPROVEMENTS.md + v1.0-experiments git tag	IMPROVEMENTS.md  , git tag 	 A4  / kalanlar 
3. Benchmark verileri	Temizlenmiş NL4Opt + IndustryOR JSONL'lerini indir, ie-eval/datasets/ altına koy, 5–10 örneği elle doğrula	ie-eval/datasets/ (boş) — adaptörler hazır: nl4opt.py , industryor.py	 Adaptör  / veri 
4. IE-Case seti	5–8 problem + veri dosyaları (csv/xlsx/sqlite) + ground truth + README	datasets/ie_case/ — adaptör hazır: ie_case.py	 Adaptör  / veri 
5. Pilot koşu	Deney konfigürasyon dosyası (config_experiments.yaml benzeri) + 20–30 problemlik uçtan uca koşu + hata ayıklama	ie-eval/runner.py (hazır), yeni config dosyası	 Yapılacak
6. MIPROv2	Train split ayır → pilot koşu → tam koşu → compiled/iesolver_mipro.json ; 2-modül kısıtlama kararı (§13.6)	scripts/optimize_mipro.py  , compiled/ (boş)	 Script  / koşu 
7. Ana koşular	Code freeze sonrası 3× koşu; baseline'lar aynı protokolle	runner.py , store.py , baselines.py — hepsi hazır	 Altyapı  / koşu 
8. Ablasyonlar	A1–A5 koşuları (A4 düzeltmesi yapıldı)	ablations.py 	 Altyapı  / koşu 
9. İkinci model	.env 'de model değişikliği + kısa pilot — kod değişikliği gerekmez	lm.py , config.py	 Hazır
10. Analiz + figürler	McNemar/bootstrap mevcut (stats.py); figür üretim scriptleri eksik (ablasyon çubuk, maliyet-doğruluk saçılım, tornado, hata dağılımı — matplotlib, PDF vektör)	ie-eval/analysis/ kısmen  , analysis/figures.py 	 Kısmen
11. Hata analizi	Otomatik error_class metrics'te var; etiketleme elle yapılır, ekstra kod gerekmez (isteğe bağlı: trace dökme yardımcı scripti)	metrics.py 	 Hazır

İş	Kodda yapılacak	İlgili dosyalar	Durum
12. Vaka çalışmaları	Rapor pipeline'ı hazır; yalnız koşu + arşivleme	report/ 	 Hazır
13. Tekrarlanabilirlik paketi	Public README (kurulum/koşum/tablo üretimi), LICENSE, Zenodo bağlama	README.md genişletme, LICENSE 	 Yapılacak

Kritik yol (kod gözüyle): İş 3 veri → İş 5 pilot → İş 6 MIPROv2 → İş 7 ana koşular. İş 10'un figür scriptleri, koşular sürerken paralel yazılabilir.